



单一应用配置实现多态运行 与迁移

云原生应用平台在平台工程理念下的演进





Content 目录

01 云原生应用平台的演进

02 单一应用配置实现多态运行与迁移

03 应用平台如何屏蔽复杂性



Part 01

云原生应用平台的演进

云原生应用平台在平台工程理念下的演进

云原生应用平台在平台工程理念下的演进

1. 以Kubernetes集群管理为核心
 2. 以Kubernetes资源视角管理应用，围绕容器管理设计
 3. 聚焦于提供容器运行环境（Kubernetes）和周边生态能力
 4. 使用声明式交付容器应用，不负责应用系统其它非容器组件
 5. 不负责编排交付基础设施资源
1. 以应用交付生命周期管理为核心
 2. 以应用视角管理应用，围绕应用管理设计
 3. 聚焦于上层应用，不负责提供运行环境
 4. 使用声明式（应用即代码、部署即代码、基础设施即代码）和不可变基础设施等技术，统一编排交付应用系统所有应用组件和基础设施资源
 5. 基于平台工程理念，屏蔽云原生和基础设施的复杂性
 6. 从前端到后端，插件式、可扩展、可自定义的框架

 RANCHER

2014年

 KUBESPHERE

2018年

 RIO

 KubeVela

2020年

 Walrus
 radius

2023年



Part 02

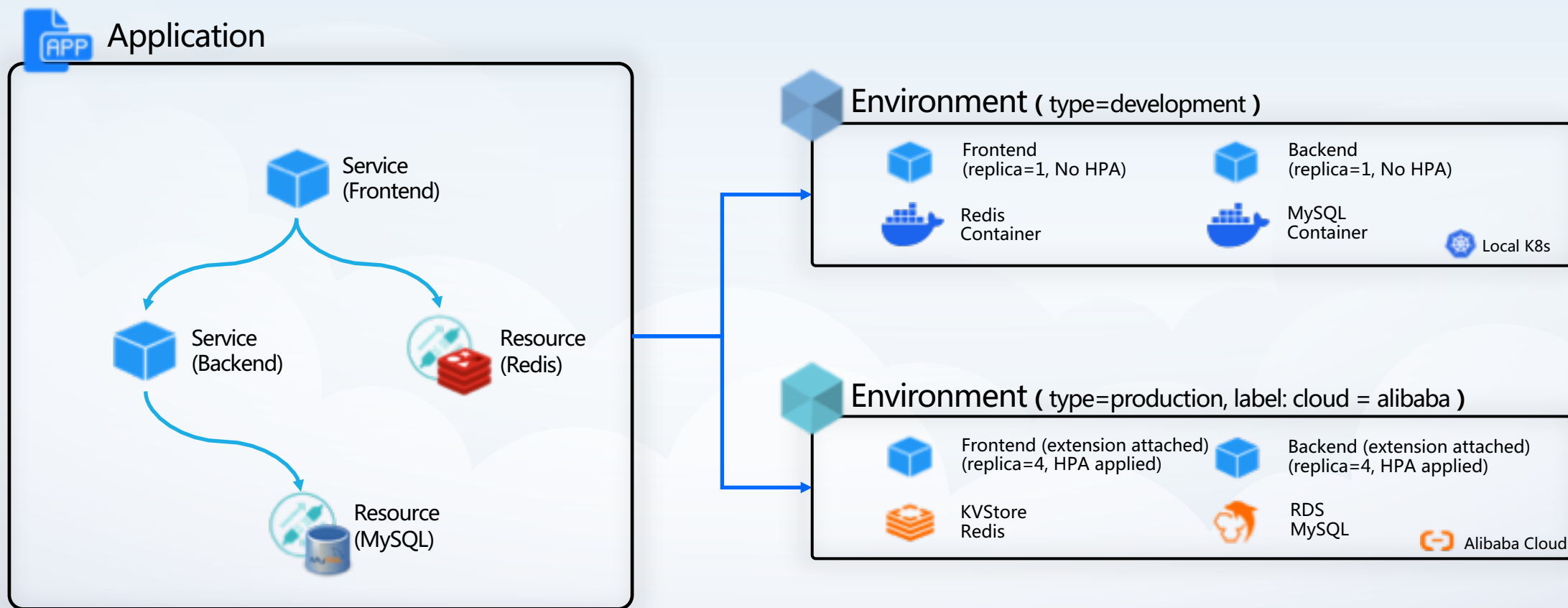
单一应用配置实现多态运行与迁移

单一配置 多态运行

单一配置 多态运行

匹配环境属性，动态扩展服务配置及实例化资源多态运行

Build Once, Run Anywhere + Configure Once, Run Polymorphically



多态运行——Resource + Resource Definition

应用侧申明所需要的Resource，包括资源类型或基础配置，无需关心具体实现方式和详细配置。

基础设施侧通过资源定义(Resource Definition)的匹配规则，为不同的环境提供不同的资源供给模式。

例如针对一个MySQL数据库，同一个资源声明在不同环境可以提供不同的运行模态：

- 容器化部署
- 虚拟机或物理机传统部署
- 云PaaS服务
-

通过Resource + Resource Definition实现研发与运维的关注点分离，研发关注Resource，运维关注用于提供Resource的Resource Definition，通过巧妙的规则匹配，实现资源供给过程的自动化，进而实现运维可控制的研发自服务。

```
# Dev focus on
resources:
- name: db
  type: mysql
  attributes:
    version: 8.0
    database: world
    username: seal
    password: ${var.db-password}
- name: cache
  type: redis
---

# Ops focus on
resource_definitions:
- name: mysql-res-def
  type: mysql
  labels: {}
  matching_rules:
  - name: mysql-for-dev
    selector:
      environment_type: development
    template:
      name: helm-mysql
      version: v0.0.1
      attributes: {}
  - name: mysql-for-prod
    selector:
      environment_type: production
    template:
      name: alicloud-rds
      version: v0.0.1
      attributes: {}
  schema: {}
```

多态运行——Service + Service Extension

通过服务扩展(Service Extension)将Service运行可能需要的高级配置、最佳实践、合规需求、运维基线等，动态地应用到不同环境的Service实例上，例如：

- HPA
- Canary
- Network Rules
- Monitoring Adapters & Rules

应用侧仅需关心应用本身的定义配置(Service)，对环境无感，无需关心在不同环境下的特定配置。

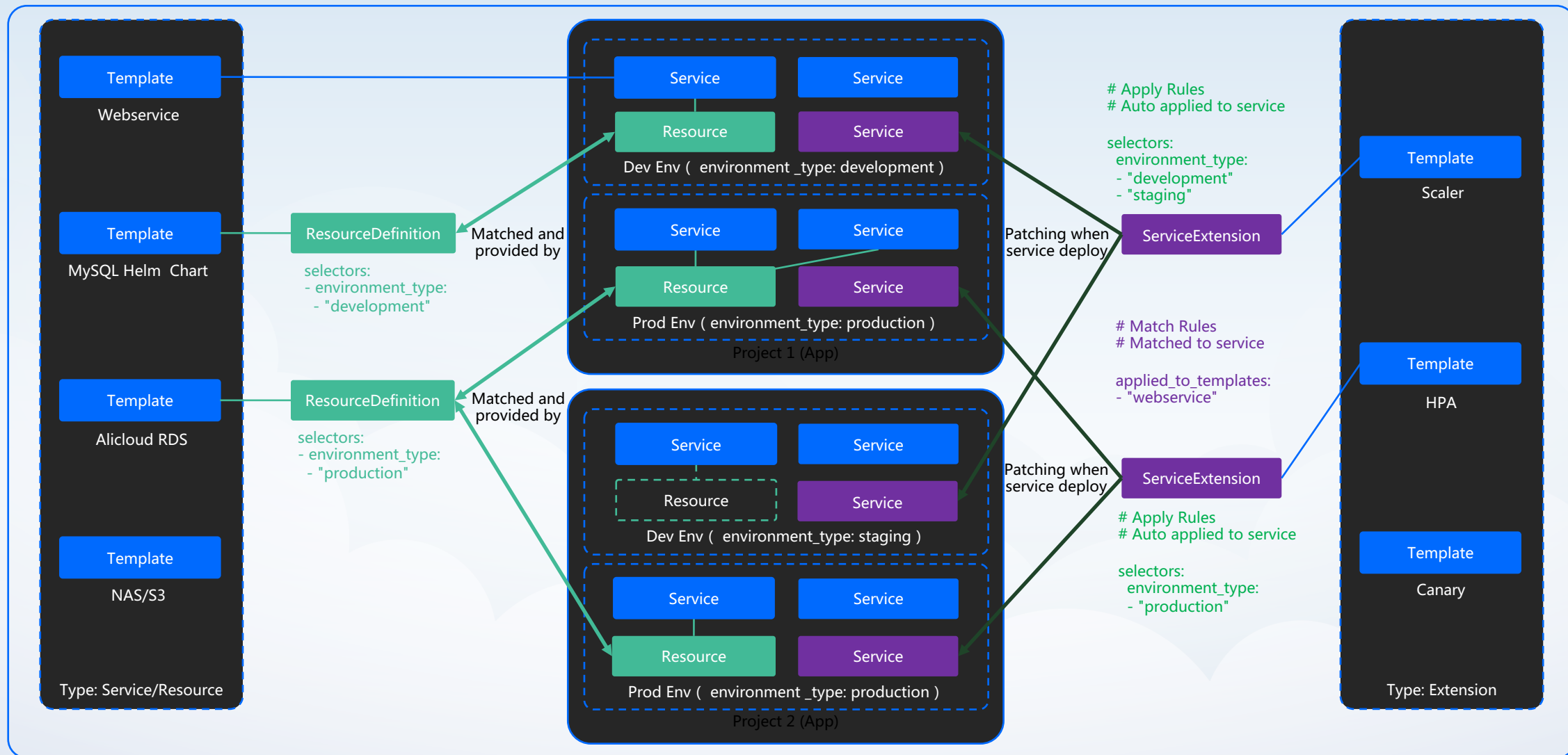
通过Service + Service Extension实现研发与运维的关注点分离，研发关注Service，运维关注用于附加运维能力的Service Extension，通过巧妙的规则匹配，实现协作过程的自动化。

对应用视角的管理人员来说，呈现和管理的仍然是完整的Service。

```
# Dev focus on
services:
- name: frontend
  template:
    name: webservice
    version: v0.0.5
  attributes:
    image: frontend:v1.1
    ports: [80]
    env:
      BACKEND_ADDR: ${resource.backend.address}
      REDIS_ADDR: ${resource.cache.address}

---
# Ops focus on
service_extensions:
- name: hpa
  applied_to_templates:
  - "webservice"
  selectors:
  - environment_type:
    - "production"
  mutation:
  - create
  - update
  template:
    name: hpa
    version: v0.0.2
  attributes:
    min_replicas: 2
    max_replicas: 10
    cpu_utilization: 60
```

单一配置 多态运行





Part 03

应用平台如何屏蔽复杂性

基于平台工程理念屏蔽复杂性，降低认知负担

基础设施资源配置细节的复杂性

云数据库 RDS

快速入门文档链接

标准创建

快速创建

免费试用

产品价格

产品详情

产品文档

产品控制台

1 基础资源

计费方式

地域

引擎

产品系列

产品类型

存储类型

购买时长 1年 75折

主节点可用区

杭州 可用区J

杭州 可用区K

杭州 可用区I

杭州 可用区O

部署方案

多可用区部署

单可用区部署

备节点可用区

存储空间

20GB

80000GB

160000GB

320000GB

100

GB

实例规格

实例规格族

通用型

通用型

通用型

通用型

通用型

通用型

通用型

网络类型

专有网络

常见问题

VPC

default (默认) | 172.16.0.0/12

共有: 11

1/1

如要创建新的专有网络, 您可以到 专有网络控制台 创建。

主节点交换机

表名大小写

不区分大小写 (默认)

区分大小写

注意: MySQL 8.0 表名大小写设置后不支持修改, 请谨慎设置。

备节点交换机

小版本升级策略

自动升级

手动升级

选择小版本

加入白名单

资源组

默认资源组

注意: 您可以将单个云账号下多个地域的多种云资源归集到不同的资源组, 方便管理。系统提供默认资源组, 您也可以 创建或管理资源组。

数据库端口

实例描述 (选项)

长度为2-255个字符, 以大小写字母或中文开头, 可包含数字, “_”或“-”

默认存储引擎

参数模板

时区

标签 (选项)

+ 添加标签 0/20

高权限账号 (选项)

立即设置

稍后设置

立即购买

第一层封装屏蔽——模板化

通过Infrastructure as Code进行模板化，屏蔽基础设施资源配置细节，降低基础设施复杂性

配置 ⓘ

Basic Advanced

Architecture ⓘ standalone

Engine Version ⓘ 8.0

Username ⓘ rdsuser 7/16

Password ⓘ 0/32

Database ⓘ world

Resources ⓘ

Class ⓘ rds.mysql.s2.large

Storage ⓘ

Class ⓘ local_ssd

Size(MB) ⓘ 20480

配置 ⓘ

Basic Advanced

Infrastructure ⓘ

Vpc Id ⓘ Kms Key Id ⓘ

Domain Suffix ⓘ Publicly Accessible ⓘ

第二层封装屏蔽——动态UI Schema

通过UI Schema定制UI表单，屏蔽基础设施资源配置细节，降低基础设施复杂性

```
1 openapi: 3.0.3
2 info:
3   title: OpenAPI schema for template terraform-alicloud-rds-mysql
4   version: v0.0.0
5 components:
6   schemas:
7     variables:
8       properties:
9         architecture:
10          default: standalone
11          description: >
12            Specify the deployment architecture, select from standalone or
13            replication.
14          enum:
15            - standalone
16            - replication
17          title: Architecture
18          type: string
19          x-walrus-ui:
20            group: Basic
21            hidden: true
22            order: 3
23          context:
24            database:
25            engine_parameters:
26            engine_version:
27            infrastructure:
28            password:
29            replication_readonly_replicas:
30            resources:
31            storage:
32            username:
33          required:
34            - infrastructure
35            - architecture
36            - engine_version
37          type: object
38          x-walrus-ui:
39            groupOrder:
40              - Basic
41              - Advanced
```

配置 ①

Engine Version *

8.0

Database ①

world

5/64

Username ①

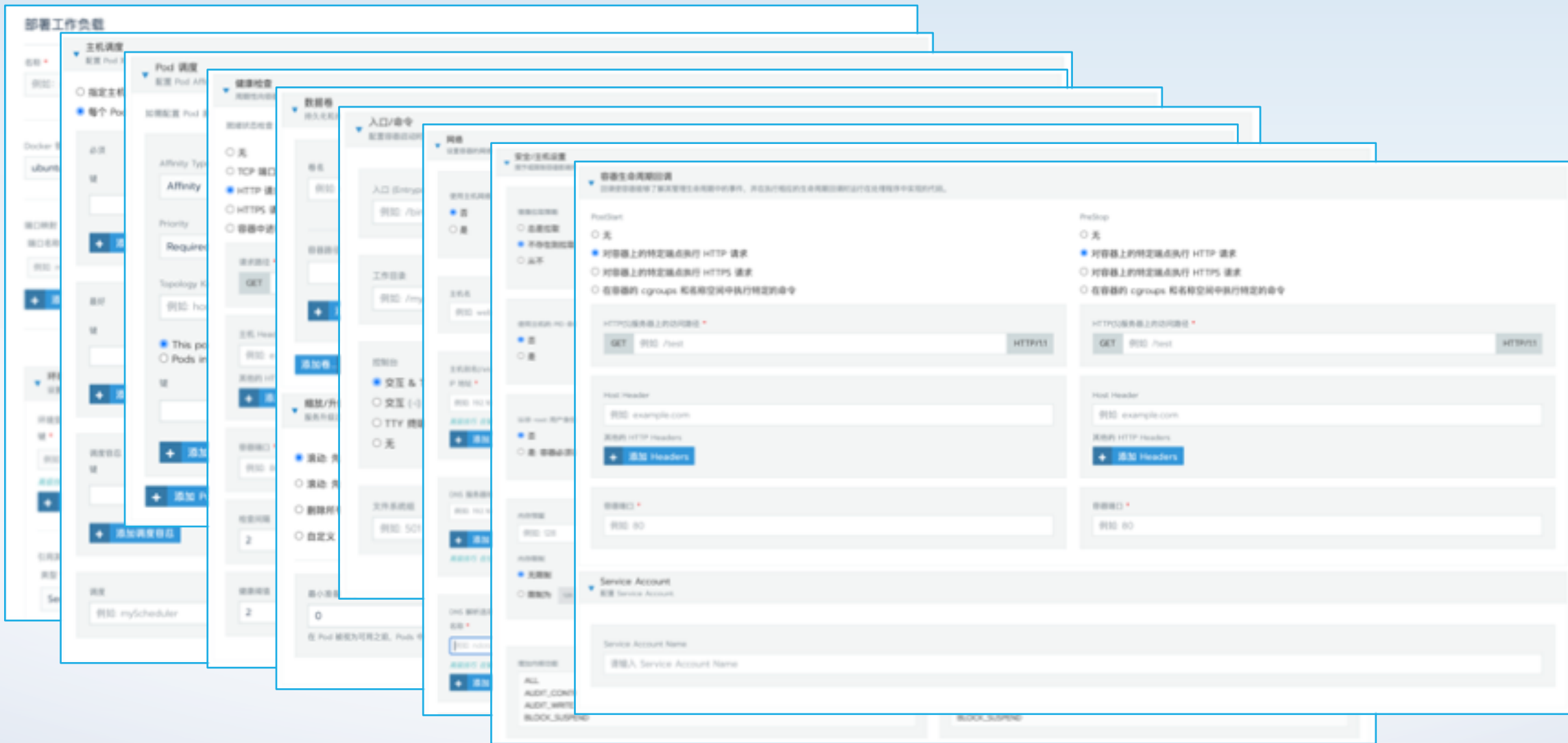
rdsuser

7/16

Password ①

0/32

云原生应用配置细节的复杂性



第一层封装屏蔽——模板化

通过Application/Deployment as Code进行模板化，屏蔽云原生应用配置细节，降低云原生复杂性

The image displays three overlapping screenshots of a configuration interface, illustrating the template-based approach to cloud-native application configuration.

- Leftmost Screenshot (Basic Tab):** Shows the 'Containers' section with an 'Image' field and a 'Ports' section with 'Internal' and 'External' port configuration options. Below this is an 'Execute' section with fields for 'Command', 'Args', 'Working Dir', 'As Group', and 'As User'.
- Middle Screenshot (Files Tab):** Shows the 'Files' section with a 'Path' field and a 'Content' field. Below this is a 'Checks' section with a 'Type' dropdown set to 'http', a 'Retries' field set to '1', and a 'Delay(Sec)' field.
- Rightmost Screenshot (Advanced Tab):** Shows the 'Infrastructure' section with 'Domain Suffix' (cluster.local) and 'Service Type' (NodePort). Below this is the 'Deployment' section with 'Replicas' (2) and 'Timeout(Sec)' (300). The 'Rolling' section shows 'Max Surge' (0.25) and 'Max Unavailable' (0.25). The 'Syntax' section has a 'Name' field and a 'Value' field.

第二层封装屏蔽——动态UI Schema

通过UI Schema定制UI表单，屏蔽云原生应用配置细节，降低云原生复杂性

```
1 openapi: 3.0.3
2 info:
3   title: OpenAPI schema for template terraform-kubernetes-containerservice
4   version: v0.0.0
5 components:
6   schemas:
7     variables:
8       properties:
9         containers:
10          description: 容器配置
11          items:
12            properties:
13              checks: 检查
14              envs: 环境变量
15              execute: 执行命令
16              files: 文件
17              image: 镜像
18              mounts: 挂载
19              ports: 端口
20              profile: 配置文件
21              resources:
22                properties:
23                  cpu:
24                    default: 0.1
25                    description: CPU核心数
26                    maximum: 1
27                    minimum: 0.1
28                    nullable: true
29                    title: Cpu(Core)
30                    type: number
31                    x-walrus-ui:
32                      type: number
33                      value: 0.1
34                  gpu:
35                    type: object
36                  memory:
37                    title: Memory(MB)
38                    type: number
39                    value: 256
40                title: Resources
41                type: object
42                x-walrus-ui:
43                  type: object
44                  x-walrus-ui:
45                    required:
46                      type: object
47                      x-walrus-ui:
48                        minItems: 1
49                        title: Containers
50                        type: array
51                        x-walrus-ui:
52                          context:
53                            deployment:
54                              infrastructure:
55                                required:
56                                  type: object
57                                  x-walrus-ui:
58                                    groupOrder:
59                                      - Basic
60                                      - Advanced
```

配置

Basic Advanced

Containers

Image

Ports

Internal

External

Protocoltcp

+

Resources

Cpu(Core)0.1

Memory(MB)256

+

Envs

Name

Value

+

Basic Advanced

Deployment

Replicas1

应用平台如何屏蔽复杂性



模板化封装

- 模板化封装实现复杂度抽象屏蔽
- 沉淀最佳实践配置
- 沉淀合规基线配置



动态UI Schema

- 动态UI Schema实现复杂度抽象屏蔽
- 按需定制UI表单，定义需要暴露的配置和预设的隐藏的配置细节



关注点分离

- 通过平台分离研发和运维角色的关注点
- 运维关注资源供给和运维能力
- 研发关注应用交付

SEAL Walrus + Appilot = AI辅助的云原生统一应用平台



基于平台工程理念的云原生统一应用平台

<https://github.com/seal-io/walrus>



IT领域的ADAS——AI驱动的应用自管理

<https://github.com/seal-io/appilot>

Walrus & Appilot 100% 开源!



应用系统、统一编排

编排部署整个应用系统，而不仅仅是容器服务



单一配置、多态运行

从开发到生产，单一应用配置，多态运行



服务资源、统一展示

展示完整应用服务及周边资源，实现全局视图



动态环境、按需伸缩

按需动态调整整个应用环境，而不仅仅是容器服务



AI驱动、高效管理

提供AI驱动的应用部署与管理，高效协助故障诊断



Seal 小助手



Seal 软件公众号



Thanks.

